ISSUE 日本語解説資料

Issue #7 バックエンドと永続化方針を決定する

GitHub で closed になっている issue 記録を、そのまま参照用 PDF に変換したもの。

Issue #7　更新: 2026/03/08　ソース: docs/portal/issues/issue-07-backend-persistence-decision.md

GitHub: https://github.com/PLAYER1-r7/MultiCloudProject/issues/7

計画 portal architecture インフラ

この ISSUE の要点
問い合わせ、会員情報、通知、管理機能の有無によって API とデータストアの必要性が変わる。

目標
MVP にバックエンドと永続化が必要かどうかを明確化する。

完了状態
[docs/portal/09_BACKEND_PERSISTEI (docs/portal/09_BACKEND_PERSISTEI を backend/persistence 判断の中心文書として扱う

1. この Issue が扱っていたこと

まず Summary と Goal から、この Issue が何を決めるためのものだったかを短く押さえる。

問い合わせ、会員情報、通知、管理機能の有無によって API とデータストアの必要性が変わる。

目標
MVP にバックエンドと永続化が必要かどうかを明確化する。

2. 完了判断の見方

Tasks、Definition of Done、Current Status の記述から、どこまで完了済みとして扱っているかを先に読む。

TASKS
7/7 項目が完了済みとして記録されている。

DEFINITION OF DONE
6/6 項目が完了済みとして記録されている。

> docs/portal/09_BACKEND_PERSISTENCE_DRAFT.md を backend/persistence 判断の中心文書として扱う / local issue file には task contract、discussion draft、resolution、evidence mapping、final review result を追加済み / 初回リリースの backend/persistence judgment は `no custom API` と `no application data persistence` で確定した / inquiry、member data、notification、CMS update の各論点は first-release deferral 条件まで含めて文書化した / AWS 初期構成に API Gateway、Lambda、DynamoDB を含めない判断は文書上で整列済みである / final checkbox review は完了し、Tasks と Definition of Done は満了と判断した / GitHub Issue 7 は最終判断に基づいて close し、Issue 7 は完了と判断する

3. 依存関係と読み進め方

Dependencies や前提 Issue がある場合は、ここで関連先を先に把握しておくと読みやすい。

依存関係

- Issue 2
- Issue 3
- Issue 4

読み方の注意

この解説書は原文の意思決定や完了状態を読みやすくするための補助資料である。最終的な正本は原文 Issue にあり、判断変更時は原文側を先に更新する。

4. 用語集

原文に残る英語や実務用語のうち、訳し切らずに補助説明を付けた方が読みやすい語をここにまとめる。

原語	日本語の扱い	補足
baseline	基準線	後続作業が参照する最小限の判断・実装・検証の出発点。
hardening	堅牢化	既存機能を壊れにくくし、運用や検証の安全性を高める作業。
workflow	ワークフロー	GitHub Actions などで定義された自動処理の流れ。
owner	責任者	意思決定または更新責任を持つ担当者。文脈により product owner などを含む。
review	レビュー	内容や実装が要件を満たすか確認する工程。
evidence	証跡	判断や完了状態の根拠になるログ、表、文書、出力結果。
mapping	対応付け	チェック項目とその根拠を結び付けて整理すること。
draft	下書き	まだ確定前だが議論やレビューの基礎として置かれた文書。
release	リリース	利用者に公開するまとまりとしての提供単位。
rollback	ロールバック	変更後の状態から以前の安定状態へ戻すこと。
close approval	クローズ承認	Issue を完了扱いとして閉じてよいという明示承認。
current status	現在の状態	Issue 文書時点での完了度合いや同期状況を示す節。
definition of done	完了条件	その Issue を完了とみなすために満たすべき条件。
task contract	タスク契約	対象、目的、スコープ、検証計画、リスクを明文化した作業契約。

5. 原文

以下に Issue 原文をそのまま掲載する。詳細な根拠、証跡、チェックリストはこの原文を参照する。

概要

問い合わせ、会員情報、通知、管理機能の有無によって API とデータストアの必要性が変わる。

目標

MVP にバックエンドと永続化が必要かどうかを明確化する。

タスク契約

```
タスク契約

メタデータ
- タスク ID: ISSUE-07
- タイトル: バックエンドと永続化方針の初期判断
- 依頼者: repository owner
- 担当者: AI agent
- 対象アプリ: portal-web
- 対象環境: planning
- 優先度: 高

目的
- 解決する問題: MVP の公開範囲に対して custom API と永続化が必要かどうかを先に切り分け、後続の AWS 初期構成判断に接続できるようにする
- 期待する価値: frontend-only で成立する範囲と backend 導入が必要になる条件を区別し、Issue 8 以降の設計前提を固定できる

スコープ
- 含むもの: 問い合わせ、会員情報、通知、CMS 的更新の要否整理; custom API 要否の判断軸整理; 保存対象と保存先候補の棚卸し
- 含まないもの: 実装、インフラ作成、認証方式の詳細設計、運用体制の確定
- 編集可能パス: docs/portal/issues/issue-07-backend-persistence-decision.md, docs/portal/03_PRODUCT_DEFINITION_DRAFT.md
- 制限パス: apps/, infra/, deployment workflows

受け入れ条件
- [x] 条件 1: MVP の前提だけで custom API と永続化の必要有無を議論開始できる状態になっている

実装計画
- 変更見込みファイル: docs/portal/issues/issue-07-backend-persistence-decision.md, docs/portal/03_PRODUCT_DEFINITION_DRAFT.md
- アプローチ: まず機能別に「本当に保存が必要か」を確認し、そのあと API の必要性と保存先候補を切り分ける
- 採用しなかった代替案と理由: backend を先に必要前提で置く案は、public-first MVP の範囲を過大評価しやすいため採らない

検証計画
- 実行するテスト: git status --short; GH_PAGER=cat gh issue view 7 --repo PLAYER1-r7/MultiCloudProject --json
number,state,title,updatedAt,url
- 確認するログ/メトリクス: TBD
- 失敗時の切り分け経路: product definition と Issue 2/3/4 の前提に戻り、MVP に持ち込んでいる動的要件が本当に必要かを再確認する

リスクとロールバック
- 主なリスク: 将来要件を先取りして backend 必須と誤判定すること
- 影響範囲: Issue 8 以降の multicloud 設計、IaC 方針、CI/CD 方針
- 緩和策: first-release scope と later-phase candidates を分けて記録する
- ロールバック手順: 必要条件が崩れたら API 必須判断を撤回し、frontend-only 前提へ戻して再評価する
```

作業項目

- [x] 問い合わせ機能の要否を確認する
- [x] 会員情報管理の要否を確認する
- [x] 通知機能の要否を確認する
- [x] CMS 的更新の要否を確認する
- [x] API の必要有無を決定する

- [x] データ保存対象を整理する
- [x] 保存先候補を整理する

完了条件

- [x] 初回リリースで custom API が必要か不要かが明示的に決定されている
- [x] 保存対象データが存在するかどうかが種類別に整理されている
- [x] 保存しない情報と、その理由が整理されている
- [x] API Gateway、Lambda、DynamoDB の要否判断に接続できる状態になっている
- [x] 問い合わせ、会員情報、通知、CMS 的更新について採否の前提が説明されている
- [x] AWS 初期構成へ反映可能な判断材料として参照できる

Issue 7 Discussion Draft

このセクションは、Issue 7 の採否判断を進めるための議論用たたき台として開始し、現在は final checkbox review に使う判断根拠を記録する。現時点の既定路線は public-first、static-first、repository-driven update であり、backend と persistence は必要条件が確認できた場合だけ導入する。

1. 現時点の前提整理

- MVP は公開ポータルであり、初回リリース時点では end-user login を前提にしない
- MVP の主要役割は「訪問者に portal の目的を伝え、次の行動へ導くこと」である
- コンテンツ更新は portal 内管理画面ではなく repository と deploy workflow 経由で行う
- Issue 4 の AWS 初期構成は S3 + CloudFront + ACM を baseline とし、API Gateway、Lambda、DynamoDB は未採用である
- よって Issue 7 の判断軸は「その未採用を崩すだけの具体的 workflow が MVP に本当にあるか」である

2. 機能別の採否たたき台

2-1. 問い合わせ機能

提案:

- 初回リリースでは portal 内送信フォームを持たず、guidance page に外部連絡先または連絡手段を記載する形を第一候補とする

この提案を支持する理由:

- MVP の価値は「案内と導線」であり、「問い合わせ処理そのもの」ではない
- フォーム受付を portal 内で完結させると、入力検証、スパム対策、保存先、通知先、障害時の再送設計が一気に必要になる
- 問い合わせ受付を外部手段へ委ねれば、初回リリースで custom API を導入せずに済む

再検討が必要になる条件:

- portal 内で問い合わせを受け付けなければ MVP の主要導線が成立しない
- 問い合わせ内容を構造化して記録し、対応状況を追跡する必要がある
- 外部フォームや既存連絡手段では運用上不十分と判断される

暫定判断:

- 現時点では「問い合わせ導線は必要、問い合わせ処理 backend は不要」を第一案とする

2-2. 会員情報管理

提案:

- 初回リリースでは会員情報や user-specific record を扱わない

この提案を支持する理由:

- Product Definition と Auth Decision はどちらも public-first かつ no end-user login を前提にしている
- 会員情報を持つ場合、認証、認可、個人データ管理、更新責任、削除方針まで同時に必要になる
- 現在の MVP page set と major scenario からは、その必要性が読み取れない

再検討が必要になる条件:

- 会員向け専用コンテンツやマイページ的機能が MVP に入る
- portal 上で user ごとに状態を保持する必要が生じる

- 利用規約、通知、申請などが user identity と結びつく

暫定判断:

- 初回リリースでは会員情報管理は不要とする

2-3. 通知機能

提案:

- 初回リリースでは portal application としての通知機能は持たない

この提案を支持する理由:

- 通知は通常、保存対象、購読対象、配信契機、配信失敗時挙動を伴う
- 現在の MVP は visitor に公開情報を見せることが中心であり、個別通知や event-driven workflow は確認されていない
- 運用通知が必要でも、それは portal の app feature ではなく monitor/deploy 側の運用設計で扱える

再検討が必要になる条件:

- visitor や member に対して portal 側から定期またはイベント通知を送る必要がある
- 問い合わせ受付や申請処理と連動した通知が MVP に必要になる

暫定判断:

- 初回リリースでは end-user 向け通知 backend は不要とする

2-4. CMS 的更新

提案:

- 初回リリースでは in-portal CMS や admin console を持たず、repository-driven update を継続する

この提案を支持する理由:

- すでに content owner と technical owner の役割、および repository-driven workflow の前提が定義されている
- CMS を導入すると auth、editor UI、保存形式、下書き管理、承認フローが追加で必要になる
- 初回リリースの page 数と更新頻度を考えると、repository-driven update のほうが構成が小さい

再検討が必要になる条件:

- 非エンジニア運用者が repository を使わずに即時更新する必要がある
- 更新頻度が高く、deploy ベース更新では運用負荷が高い
- 下書き、承認、公開予約などの editorial workflow が必要になる

暫定判断:

- 初回リリースでは CMS backend は不要とする

3. 保存対象データの棚卸し たたき台

初回リリース時点の整理案:

データ種別	初回リリースで保存するか	現時点の理由	保存先候補
portal の公開コンテンツ	yes	ただし application database ではなく repository 上の source と build artifact として扱う	Git repository, S3
visitor の問い合わせ本文	no	portal 内送信フォームを前提にしないため	不要
visitor の会員情報	no	no login, no member workflow のため	不要
notification 購読情報	no	notification feature を持たないため	不要
CMS 下書きや承認状態	no	repository-driven update を採用するため	不要

データ種別	初回リリースで保存するか	現時点の理由	保存先候補
access log や delivery log	yes	ただし app feature の永続化ではなく運用・監視証跡として必要になりうる	CloudFront/S3/monitoring side

補足:

- 「保存しない」は「その情報が無価値」という意味ではなく、「MVP の app responsibility としてまだ持たない」という意味である
- repository と object storage に静的 asset を置くことは persistence ではあるが、Issue 7 で主に判断したいのは application data persistence の要否である

4. API 必要条件の判断フレーム

次のどれかが yes なら custom API を再検討する。

1. portal 内で受けた入力を server-side で検証、変換、保存、配信する必要があるか
2. user identity に応じて返す内容や許可を変える必要があるか
3. repository-driven update では間に合わない運用操作を portal 自体に持たせる必要があるか
4. 外部サービス連携を client-side だけで安全に扱えないか

現時点の仮回答:

- 1: no
- 2: no
- 3: no
- 4: no

暫定結論:

- 初回リリースでは custom API は不要とする案が最も整合的である

5. AWS 初期構成への接続

上記の暫定判断を採る場合、AWS 初期構成への影響は次の通り。

- API Gateway: 不要
- Lambda: 不要
- DynamoDB: 不要
- S3 + CloudFront + ACM の baseline を維持する
- 監視対象は application transaction よりも static delivery と availability を中心に設計する

逆に、問い合わせフォーム、会員情報、通知、CMS のどれかを MVP に入れる判断へ変わる場合は、Issue 4 の非採用理由を再評価する必要がある。

6. この場で確認したい論点

- 問い合わせは「連絡導線があれば十分」か、それとも「portal 内で送信完了まで提供する」必要があるか
- 公開 visitor 以外に、MVP 時点で識別された user を扱う要件は本当にないか
- 通知は portal feature として必要か、それとも運用連絡だけで十分か
- content update は repository-driven update のままで回るか、それとも non-technical editor 向け UI が必要か
- 「保存対象」として app data と operational log を分けて扱う整理で問題ないか

7. 議論開始用の暫定結論

- 初回リリースは static-first を維持し、custom API は導入しない
- application database は導入しない
- 問い合わせは guidance または外部導線で扱い、portal 内 workflow にはしない
- 会員情報、通知、CMS 的更新は later phase 候補として整理し、MVP から外す
- もし上記のどれかを MVP 必須と判断する場合、その時点で API Gateway、Lambda、DynamoDB の採否を再度開く

8. Issue 7 の結論案

提案する結論文:

- 初回リリースの portal は public-first かつ static-first の前提を維持し、custom API と application data persistence は導入しない
- 問い合わせは portal 内送信機能としては扱わず、guidance page から外部連絡手段または外部 workflow へ誘導する

- 会員情報管理、end-user 向け通知、CMS 的更新は MVP 要件に含めず、later phase の候補として分離する
- 保存対象として扱うのは repository 上のコンテンツ source、build artifact、配信や監視の運用証跡までとし、visitor や member に紐づく application record は持たない
- したがって、AWS 初期構成に API Gateway、Lambda、DynamoDB は含めず、S3 + CloudFront + ACM を baseline とする

この結論で満たせること:

- Issue 2 の public guidance 中心の MVP と矛盾しない
- Issue 3 の no end-user login 前提と整合する
- Issue 4 の AWS baseline 非採用判断を維持できる
- Issue 8 以降で「何を cloud-portable constraint として残すか」を static delivery 前提で整理できる

9. 今回採らない案と理由

採らない案 1: 問い合わせフォームを MVP に含める

- 理由: 入力検証、送信先、保存、通知、スパム対策が連鎖し、MVP の範囲を超えて backend 導入圧力が高くなるため

採らない案 2: 会員情報を先に保存できるようにする

- 理由: login 不要という既定方針と衝突し、認証・認可・個人データ管理の設計を前倒ししてしまうため

採らない案 3: CMS 的更新 UI を初回から入れる

- 理由: 現時点では repository-driven update で運用可能であり、editor UI と承認 workflow を足す必要性が確認されていないため

10. 結論を確定するための最終確認

次の 4 点に異論がなければ、Issue 7 は上記結論案で固めてよい。

1. 問い合わせは portal 内処理ではなく、外部導線で十分である
2. MVP 時点で識別された user record を保存する必要はない
3. end-user 向け通知機能は MVP に不要である
4. content update は repository-driven update のままで運用開始できる

Resolution

Issue 7 の判断結果は次の通りとする。

- 初回リリースでは custom API を導入しない
- 初回リリースでは visitor または member に紐づく application data persistence を導入しない
- 問い合わせは guidance page から外部連絡手段または外部 workflow へ誘導し、portal 内送信 workflow にはしない
- 会員情報管理、end-user 向け通知、CMS 的更新は MVP に含めず、later phase の候補として扱う
- AWS 初期構成は S3 + CloudFront + ACM を baseline とし、API Gateway、Lambda、DynamoDB は含めない

根拠文書:

- docs/portal/09_BACKEND_PERSISTENCE_DRAFT.md
- docs/portal/03_PRODUCT_DEFINITION_DRAFT.md
- docs/portal/04_MVP_SCOPE_DRAFT.md
- docs/portal/05_AUTH_DECISION_DRAFT.md
- docs/portal/06_AWS_ARCHITECTURE_DRAFT.md

Issue 8 以降への引き継ぎ事項:

- backend や persistence を前提にした設計は行わず、static delivery を baseline にする
- backend 再検討条件は「portal 内入力処理」「user-specific state」「server-side authorization」「in-portal operational editing」のいずれかが validated requirement になった場合に限る
- monitoring、security、test、rollback は application transaction ではなく static delivery と public guidance flow を中心に設計する

証跡対応表

The tables below identify the evidence used for final checkbox review and should remain aligned with the checked state above.

作業項目対応表

Checklist item	Primary evidence section	Why this is the evidence	Review state
問い合わせ機能の要否を確認する	2-1. 問い合わせ機能, 3. 保存対象データの棚卸し たたき台, and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections define the inquiry handling recommendation, explain why in-portal submission is deferred, and record the final resolution to use external guidance instead of backend workflow.	Accepted for final review
会員情報管理の要否を確認する	2-2. 会員情報管理, 3. 保存対象データの棚卸し たたき台, and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections state that no user-specific records are required for the first release and connect that judgment to the no-login MVP model.	Accepted for final review
通知機能の要否を確認する	2-3. 通知機能 and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections explain why end-user notification backend logic is unnecessary for the first release and record the final deferral to later phase.	Accepted for final review
CMS 的更新の要否を確認する	2-4. CMS 的更新 and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections define repository-driven update as the first-release operating model and reject in-portal CMS for the MVP.	Accepted for final review
API の必要有無を決定する	4. API 必要条件の判断フレーム, 5. AWS 初期構成への接続, and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections evaluate the trigger conditions for a custom API, answer them as no for the current release, and translate that decision into AWS service implications.	Accepted for final review
データ保存対象を整理する	3. 保存対象データの棚卸し たたき台 and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	The table identifies which data classes exist, which are intentionally not stored as application data, and how static content and operational evidence are treated separately.	Accepted for final review
保存先候補を整理する	3. 保存対象データの棚卸し たたき台, 5. AWS 初期構成への接続, and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections identify repository, S3, and monitoring-side storage as the only relevant first-release storage targets while excluding DynamoDB and other application persistence.	Accepted for final review

完了条件対応表

Checklist item	Primary evidence section	Why this is the evidence	Review state
初回リリースで custom API が必要か不要かが明示的に決定されている	4. API 必要条件の判断フレーム, 8. Issue 7 の結論案, and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections explicitly converge on no custom API for the first release and explain when that decision would be reopened.	Accepted for final review
保存対象データが存在するかどうかが種類別に整理されている	3. 保存対象データの棚卸し たたき台 in docs/portal/issues/issue-07-backend-persistence-decision.md	The table separates public content, inquiry text, member data, notification subscriptions, CMS state, and operational logs by storage decision and rationale.	Accepted for final review

Checklist item	Primary evidence section	Why this is the evidence	Review state
保存しない情報と、その理由が整理されている	2. 機能別の採否たたき台, 3. 保存対象データの棚卸し たたき台, and 9. 今回採らない案と理由 in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections explain both the non-adopted workflows and the reasons why their associated data should not be stored in the MVP.	Accepted for final review
API Gateway、Lambda、DynamoDB の要否判断に接続できる状態になっている	5. AWS 初期構成への接続 and Resolution in docs/portal/issues/issue-07-backend-persistence-decision.md	These sections translate the product-level backend decision into explicit non-adoption for API Gateway, Lambda, and DynamoDB in the AWS baseline.	Accepted for final review
問い合わせ、会員情報、通知、CMS 的更新について採否の前提が説明されている	2. 機能別の採否たたき台 in docs/portal/issues/issue-07-backend-persistence-decision.md	The four subsections record the recommended posture, supporting rationale, and reopen conditions for each capability area.	Accepted for final review
AWS 初期構成へ反映可能な判断材料として参照できる	5. AWS 初期構成への接続, Resolution, and docs/portal/09_BACKEND_PERSISTENCE_DRAFT.md	These sections make the first-release backend and persistence decision explicit enough to guide downstream AWS, IaC, CI/CD, monitoring, and security planning.	Accepted for final review

Final Review Rule For Issue 7

- The presence of mapped evidence is not by itself enough to check the box.
- Final checkbox review should confirm that the mapped section still reflects the latest agreed wording.
- If a mapped section changes materially, the table should be rechecked before marking the checklist item complete.
- If the local issue definition changes after the last remote sync, the GitHub issue body should be synced again before close or any equivalent final-state transition.

最終レビュー結果

Final checkbox review completed against the latest wording in docs/portal/issues/issue-07-backend-persistence-decision.md and docs/portal/09_BACKEND_PERSISTENCE_DRAFT.md. The table below records the document-level validation judgment. Human close approval remains a separate action.

Checklist area	Final judgment	Evidence basis
Inquiry handling decision	Satisfied	2-1. 問い合わせ機能, 3. 保存対象データの棚卸し たたき台, and Resolution confirm that the MVP keeps inquiry guidance but does not introduce in-portal submission or backend processing.
Member-data decision	Satisfied	2-2. 会員情報管理, 3. 保存対象データの棚卸し たたき台, and Resolution confirm that the first release does not require user-specific records or member management.
Notification decision	Satisfied	2-3. 通知機能 and Resolution confirm that end-user notification workflows are deferred and do not justify backend introduction in the MVP.
CMS update decision	Satisfied	2-4. CMS 的更新 and Resolution confirm that repository-driven updates remain the operating model and no in-portal CMS is required.

Checklist area	Final judgment	Evidence basis
API adoption decision	Satisfied	`4. API 必要条件の判断フレーム`, `5. AWS 初期構成への接続`, and `Resolution` converge on no custom API for the first release and define the reopen triggers.
Persistence scope decision	Satisfied	`3. 保存対象データの棚卸し たたき台` and `Resolution` separate static content and operational evidence from application data persistence and exclude visitor or member records.
AWS baseline readiness	Satisfied	`5. AWS 初期構成への接続`, `Resolution`, and docs/portal/09_BACKEND_PERSISTENCE_DRAFT.md connect the product-level decision to non-adoption of API Gateway, Lambda, and DynamoDB.

プロセスレビュー記録

- The Task Contract now exists in this file and captures scope, validation, and rollback, but it was added during the review-hardening flow rather than before the earliest discussion work began.
- The current record is sufficient for auditability of the final decision, but it should be treated as a retrospective contract record rather than proof that the full protocol order was followed from the first minute.
- Final review output is now separated from evidence mapping so later reviewers can distinguish completion judgment from source locations.

現在の状態

- docs/portal/09_BACKEND_PERSISTENCE_DRAFT.md を backend/persistence 判断の中心文書として扱う
- local issue file には task contract、discussion draft、resolution、evidence mapping、final review result を追加済み
- 初回リリースの backend/persistence judgment は `no custom API` と `no application data persistence` で確定した
- inquiry、member data、notification、CMS update の各論点は first-release deferral 条件まで含めて文書化した
- AWS 初期構成に API Gateway、Lambda、DynamoDB を含めない判断は文書上で整列済みである
- final checkbox review は完了し、Tasks と Definition of Done は満了と判断した
- GitHub Issue 7 は最終判断に基づいて close し、Issue 7 は完了と判断する

依存関係

- Issue 2
- Issue 3
- Issue 4